

LED - Ligando dados, pessoas e conhecimento

ELT para base pública de corridas de táxi em Nova York

Data de Inicio	13/7/2026
Data de Fim	28/7/2026

01 Visão geral do desafio

A equipe de criação do aplicativo U-Easy tem como intuito o desenvolvimento de uma aplicação mobile que integre facilidades para vida cotidiana. Na etapa atual, eles têm estudado a mobilidade de grandes centros por meio de arquivos privados e pagos de outras instituições sobre corridas de táxi. Contudo, não possuem expertise em **análise e engenharia de dados** e buscam mão de obra terceirizada para reforçar sua capacidade.

Você e sua equipe buscam essa oportunidade e para isso eles disponibilizaram inicialmente arquivos públicos, da mesma temática num recorte de tempo específico, para avaliar a capacidade de vocês de **transformá-los em uma base confiável para [consultas analíticas](#) que informem de maneira precisa, confiável e clara insights e estatísticas**.

Os dados chegam em **formatos de arquivo (Parquet e CSVs)**, com códigos, datas, valores monetários, localizações e entre outros campos com **inconsistências**. A missão do grupo é construir um processo **ELT** completo usando somente SQL (equipes 1-17) ou uma outra tecnologia de transformação, como PySpark (equipe 18 com pós-graduandos).

Princípio obrigatório

A comunicação deve ocorrer pelo canal do Discord disponibilizado para que a LED tenha oportunidade de acompanhar, sanar dúvidas e avaliar o desenvolvimento.

O que deve existir ao final:

- **À nível de dados e processamento:**

1. Um [banco analítico](#) reproduzível (compreenda reproduzível como sendo capaz facilmente de qualquer outro ser capaz de construí-lo a partir do arquivo bruto novamente, havendo uma ordem de execução clara para isso);
2. Dados brutos preservados em tabela (para fins de auditoria e rastreabilidade);
3. Verificações de qualidade armazenadas para justificar as mudanças (Ex.: Verificou nulidade? Duplicidade? Tenha isso armazenado em tabela ou as consultas que foram utilizadas para essa verificação);
4. Registro das transformações realizadas (Seja em notebook, em arquivo .sql ou outro). As transformações devem ser por SQL para as equipes 1-17 e por outra tecnologia para os pós graduandos (equipe 18);
5. Dados tratados num modelo dimensional;
6. [Consultas](#) capazes de responder perguntas de negócio.

- **Outras entregas:**

1. Repositório organizado e privado (será necessário adicionar emails de membros da LED);
2. Documentação que embase as decisões tomadas, as alternativas descartadas e o papel de cada membro da equipe;
3. Slides para a apresentação e a apresentação em si. Todos os integrantes devem participar. A LED poderá escolher qualquer membro para explicar uma decisão, interpretar uma consulta e etc.

Objetivos de aprendizagem

1. Entender, na prática, como dados externos entram em um ambiente analítico e atravessam diferentes camadas até se tornarem úteis;
2. Investigar problemas de qualidade, definir regras de limpeza e justificar decisões, ainda que subjetivas, sem tentar excluir manualmente no arquivo de origem dados inconvenientes ou realizar uma exclusão de forma escalável, mas sem critério que embase;
3. Desenvolver um olhar investigativo e crítico para tomar as decisões arquitetônicas e de transformação, envolvendo inclusive o contexto em si;
4. Distinguir um [banco transacional](#) de um [banco analítico](#) e aplicar princípios de modelagem dimensional;
5. Escrever SQL (ou código para equipe 18) organizado, legível, testável e capaz de ser executado novamente do início ao fim;
6. Ter resultados ao final que vão apoiar pessoas não-técnicas (ex.: [consultas](#) que poderiam alimentar facilmente dashboards);
7. Desenvolver autonomia para pesquisar conceitos, consultar documentação e aprender ferramentas novas;
8. Realizar esse processo em equipe, sabendo comunicar-se internamente e externamente (com a LED, por exemplo);
9. Documentar para leitura que detenha conhecimento e que justifique as tomadas de decisão no [ELT](#);
10. Desenvolver soft skills relacionadas ao esforço, a resiliência, a constância e **o compartilhamento de conhecimento**;
11. Ao final, apresentar de forma coesa e que compreenda-se o raciocínio facilmente tido.

OLTP não é o objetivo deste projeto

Aspecto	OLTP (Transacional)	OLAP (analítico)
Objetivo	Registrar uma operação corretamente	Poder analisar muitas operações e encontrar padrões
Arquitetura típica	Tabelas bem separadas (normalizadas), muitas escritas e atualizações	Fatos e dimensões, leituras, filtros e agregações

O modelo final deve ser construído para análise. **Não basta importar o arquivo para uma única tabela e executar alguns SELECTs.**

02 Base de dados e ambiente

Base: NYC Yellow Taxi Trip Records

O desafio utilizará os registros de corridas dos táxis da cidade de Nova York em janeiro de 2024. Cada registro representa uma viagem e contém datas e horários, zonas de embarque e desembarque, distância, quantidade de passageiros, forma de pagamento e componentes do valor cobrado. Essa base combina dados temporais, geográficos, categóricos e financeiros, além de

códigos e valores que exigem investigação. Além da descoberta do contexto desses campos, a fim de torná-los mais legíveis e padronizados, há inconsistências nos mesmos.

Recurso	Uso
Yellow Taxi, Janeiro de 2024 (Parquet)	Fonte principal de viagens. Deve ser carregada sem edição manual (até porquê Parquet sequer foi feito para isso)
Taxi Zone Lookup Table (CSV)	Tabela de referência para traduzir códigos de localização em zona, <i>borough</i> (“bairros” de NY) e área de serviço. Pode ser incorporada para enriquecimento do modelo.
Dicionário de dados, Yellow Taxi	Referência para compreender o significado dos campos.

Ambiente recomendado

1. DuckDB, executado pela linha de comando ou por uma interface gráfica compatível.
2. GitHub para versionamento, colaboração e submissão do projeto;
3. Ferramentas de diagramação são permitidas para documentar o modelo, caso deseje. Mas nenhuma delas pode transformar os dados no lugar do SQL;
4. Scripts auxiliares poderão ser utilizados apenas para **inicialização** do ambiente ou execução automatizada dos arquivos SQL. A lógica de limpeza, transformação e modelagem deverá permanecer em SQL.

O DuckDB foi escolhido por ser um banco voltado a cargas analíticas e por consultar arquivos Parquet diretamente com SQL, além de estar na “crista da onda” por ser muito performático no que propõe-se. Consulte a [página oficial do DuckDB](#) e a [documentação de leitura de Parquet](#)

03 Escopo técnico obrigatório

A organização das camadas e os nomes das tabelas são decisões do grupo; os resultados mínimos abaixo são obrigatórios.

1	Extração e carga dos dados brutos Carregar os arquivos de origem para o banco usando SQL (equipes 1-17) ou alguma outra tecnologia (equipe 18 de pós graduandos), preservando uma representação fiel do que foi recebido. Não é permitido abrir o arquivo em planilha ou similar, corrigir valores manualmente e salvar uma versão “limpa”, isso não é ETL ou ELT.
2	Perfilamento dos dados Investigar tipos, volume, valores nulos, distribuições, domínios, cardinalidades , possíveis duplicidades, valores improváveis e o que mais avaliarem como pertinente. As descobertas devem orientar as regras de tratamento, as mais relevantes devem ser armazenadas em tabelas com as estatísticas ou, ao menos, documentadas de como impactaram a transformação. Além disso, as consultas que foram realizadas para essas verificações devem ser disponibilizadas.

3

Camada de tratamento

Padronizar tipos, nomes, categorias, valores, calcular [atributos](#) necessários, lidar com a nulidade, duplicatas, tipagens indevidas e entre outros processos que **avaliarem como pertinente. Toda transformação deve ter justificativa e ser disponibilizada como foi aplicada.**

4

Modelo dimensional OLAP

Construir um modelo analítico com pelo menos **uma tabela de fato e três dimensões. O grupo deve declarar a granularidade da fato, definir [chaves](#) e explicar por que cada [atributo](#) pertence à tabela escolhida.**

5

Camada de consumo analítico

Entregar [consultas](#) que respondam perguntas relevantes sobre demanda, tempo, localização, pagamentos, preços ou eficiência das viagens. Os resultados precisam usar o modelo final, e não consultar diretamente o arquivo bruto.

Princípio obrigatório

O processo deve ser [ELT](#): primeiro os dados externos são carregados no ambiente analítico; depois são transformados com SQL (ou outra tecnologia, no caso da equipe 18) dentro desse ambiente. **A camada [bruta](#) não deve ser substituída silenciosamente por uma versão corrigida.**

Condições que comprometem severamente a avaliação

1. Pipeline que não executa a partir dos dados originais ou depende de correções manuais não documentadas;
2. Transformações realizadas em Python, R (equipes 1-17), planilhas, notebooks ou ferramentas de BI. A equipe 18 não deve realizar transformações com apenas SQL;
3. Modelo final composto apenas por uma cópia da tabela [bruta](#), sem modelagem analítica;
4. Exclusão de dados sem justificativa ou possibilidade de auditoria;
5. Resultados apresentados sem SQL correspondente ou sem possibilidade de reprodução;
6. Uso de LLM em desacordo com as regras da seção 5.

04 Requisitos de limpeza e modelagem

A avaliação considera a qualidade do raciocínio, não a quantidade de tabelas ou de linhas de SQL.

Limpeza e rastreabilidade

1. As regras devem nascer do [perfilamento](#) e da documentação da fonte, não apenas de intuição;
2. Valores raros não são automaticamente erros. O grupo deve diferenciar o que é incomum, desconhecido, inválido e ausente;
3. Registros removidos ou excluídos de análises devem ser explicados;

4. Conversões de tipo precisam tratar falhas de forma consciente. Uma conversão que quebra a transformação não é a solução
5. Métricas para transformação ou criação de novas colunas devem ser explicáveis;
6. Se eu repito a execução não posso multiplicar dados ou alterar resultados sem ter tido mudanças na fonte.

Modelagem dimensional

1. Definir explicitamente a granularidade da tabela fato;
2. Separar campos usados para medida, para [chaves](#) e para [atributos descritivos](#) de acordo com o esquema estrela;
3. Decidir quando uma [chave](#) natural é suficiente e quando uma [chave](#) substituta melhora o modelo;
4. [Representar tempo e localização de forma que consultas frequentes não dependam de construir uma lógica repetida para informações que poderia ser armazenadas previamente](#);
5. Evitar copiar para a fato textos e descrições que se repetem em milhões de linhas sem justificativa;
6. O modelo deve favorecer leitura e [agregação](#) sem separar excessivamente tabelas.

Perguntas analíticas

Exemplos (não apenas essas ou exatamente dessa forma) de perguntas que devem ser respondidas com consultas:

Perspectiva	Exemplos de direção, não são perguntas prontas
Tempo	Variação por dia da semana, hora, período do dia ou duração.
Geografia	Fluxos entre zonas, áreas de maior origem/destino ou diferenças entre regiões, destinos mais frequentes em diferentes granularidades (top 10, top 5)
Financeiro	Composição do valor, gorjetas, pedágios, tarifas e formas de pagamento.
Operação	Distância, velocidade aproximada, ocupação ou viagens fora do padrão.

05 Regras de uso de LLM e integridade

IA pode apoiar a aprendizagem; não pode substituir o trabalho intelectual que o desafio pretende avaliar.

Regra central

É terminantemente proibido fazer todo o projeto com uma LLM, entregar scripts gerados integralmente por IA ou copiar uma solução que o grupo não consegue explicar, testar e modificar.

Uso permitido: LLM como tutora	Uso proibido: LLM como executora do projeto
Pedir explicações sobre ETL , ELT , OLAP , fatos, dimensões, uma função SQL ou qualquer outro conceito.	Pedir “faça o projeto completo” ou “gere todos os scripts” e apenas copiar o resultado.
Solicitar ajuda para interpretar uma mensagem de erro ou revisar um trecho pequeno.	Solicitar a modelagem final sem realizar perfilamento e sem justificar o grão .
Comparar duas abordagens e depois tomar uma decisão própria e documentada.	Enviar a base inteira para a IA decidir sozinha quais registros apagar.
Gerar termos de pesquisa, perguntas de estudo ou exemplos mínimos isolados.	Copiar consultas, testes ou documentação que nenhum integrante consegue explicar.
Revisar clareza textual do documento, mantendo conteúdo e decisões do grupo.	Ocultar o uso de IA ou declarar como autoria própria uma solução não compreendida.
Registrar o uso, conferir cada resultado e conseguir refazer sem a ferramenta.	Usar outra linguagem ou ferramenta para transformar dados e apresentar apenas o SQL resultante.

06 Apoio para pesquisa

O enunciado oferece direção, não uma regra. Pesquisar documentação faz parte do desafio. Abaixo são sugestões que devem trazer bons resultados, mas não implicam na forma que a solução deve ser construída.

Como usar esta seção

Escolha os tópicos conforme os problemas encontrados. Não tente estudar tudo antes de abrir os dados: investigue, formule perguntas, pesquise e volte ao projeto.

Quando surgir esta dúvida...	Sugestão de pesquisa (em inglês e português)	Questão que o grupo deve responder
Como organizar o fluxo?	ETL vs ELT ; raw , staging e curated layers ; pipeline com SQL, ingestão de parquet em SQL	O dado é transformado antes ou depois de entrar no banco?
Como desenhar o banco?	OLTP vs OLAP; dimensional modeling; star schema	O modelo favorece operações ou análise?
Qual é a fato?	fact table grain; measures and dimensions	O que exatamente uma linha da fato representa?
Como tratar códigos?	lookup table ; domain values ; unknown member	Código ausente é erro, desconhecido ou categoria válida?

Quando surgir esta dúvida...	Sugestão de pesquisa (em inglês e português)	Questão que o grupo deve responder
Como investigar a base?	SQL data profiling ; cardinality; null distribution, duplicidade, tipagem, distribuição, outliers, média, moda, entender grão e granularidade (rollup e drill down)	Quais evidências sustentam cada regra?
Como testar?	SQL data quality tests; uniqueness; referential integrity, consultas analíticas	Como o pipeline prova que produziu dados confiáveis?
Como reexecutar?	idempotent SQL pipeline ; incremental load x full load	Rodar duas vezes gera o mesmo resultado?
Como tratar códigos e classificações?	lookup table ; domain values ; unknown member , distribuição , nulos x desconhecido; zero discreto	Código ausente é erro, desconhecido ou categoria válida? Está sendo diferenciando esses cenários?
Como escrever em SQL para cenários complexos? Como modelar com SQL?	star schema , CTE ; OR REPLACE, TRY_CAST ; CASE; window functions; date functions; CREATE TABLE AS SELECT (cta), generate series. WITH, range(), cast, strptime, alias, date as integer for pk and fk, role playing dimension, junk dimension, outrigger dimension, dimensão degenerada , sequences for id	A consulta é clara e lida com casos inesperados?
Como auditar?	data lineage ; rejected records ; reconciliation	É possível explicar o que aconteceu com cada grupo de registros?

Perguntas-guia antes de modelar

1. O que uma linha do arquivo representa? Essa representação continuará igual na tabela fato?
2. Quais campos são [medidas](#) que podem ser somadas, calculadas ou comparadas para consultas analíticas?
3. Quais descrições se repetem e deveriam ser consultadas por meio de uma dimensão?
4. Que valores parecem impossíveis? Que valores apenas parecem estranhos?
5. Quais decisões alteram contagens, valores totais ou conclusões analíticas?
6. Como alguém externo descobriria essas decisões tomadas?

Links interessantes

 1-Introdução BI, DW e Esquema Estrela

 2-Tabelas de dimensões e de fatos, tipos de medidas e tipos de tabelas de fatos.mp4

3-Barramento do DW, Dimensões e fatos conformados e projeto básico do DW.mp4

4-Projeto avançado do DW.mp4

5-Estendendo o esquema estrela e capturando o histórico dos dados

6-OLAP

DW_OLAP.pptx

11-SQL.pptx

07 Condução sugerida do processo seletivo

A sequência abaixo equilibra autonomia, tempo para pesquisa e validação individual do aprendizado. Contudo, é apenas uma sugestão genérica.

Momento	Atividade	Resultado esperado
Dias 1–3	Exploração da fonte, leitura do dicionário e perfilamento , compreender a base. Delegação de responsabilidade.	Hipóteses, problemas encontrados e proposta de granularidade da tabela de fato.
Dias 4–9	Carga, tratamento e primeira versão do modelo dimensional. Documentação contínua das escolhas	Pipeline executável e modelo mínimo.
Dias 9–11	Testes, reconciliação e consultas analíticas.	Qualidade mensurada e perguntas respondidas.
Dias 12–14	Refino, documentação, compreensão individual e apresentação.	Entrega reproduzível e defesa preparada.
Dúvidas com a LED	Sob demanda do grupo.	Maior alinhamento antecipadamente.

Checklist final p/equipe

- ☐ O banco pode ser reconstruído do zero usando apenas as instruções do repositório.
- ☐ A fonte [bruta](#) foi armazenada e nenhuma limpeza dependeu de edição manual.
- ☐ O [grão da fato](#) está escrito em uma frase clara.
- ☐ As dimensões possuem propósito analítico e [chaves](#) coerentes.
- ☐ As regras de qualidade têm consultas e resultados interpretáveis.

- ☐ As análises consultam o modelo final como esperado
- ☐ As transformações dos dados estão justificadas.
- ☐ Todos os integrantes conseguem explicar o projeto.
- ☐ O uso de LLM foi compatível com as regras e registrado com transparência.

Mensagem aos participantes

Não é esperado que vocês já saibam tudo, que entreguem algo 100% correto ou completo. É esperado que investiguem, façam escolhas conscientes, testem o que construíram e sejam capazes de explicar o caminho percorrido. É esperado também vontade de verdade e envolvimento com o problema, com a equipe, com os assuntos. Engenharia de dados começa quando o dado deixa de ser apenas um arquivo e passa a ser algo legível que traz impacto.

08 Glossário

Termos técnicos usados ao longo deste documento, organizados na ordem em que aparecem.

Termos da seção 01 : Visão geral

ELT (Extract, Load, Transform): Os dados são primeiro carregados no ambiente analítico do jeito que chegaram (extract + load) e só depois transformados, já dentro do banco, usando SQL. Diferente do ETL, onde a transformação acontece antes da carga, fora do banco.

ETL (Extract, Transform, Load): Os dados são transformados antes de entrar no banco de destino. Contraste direto com ELT; o desafio proíbe esse padrão porque exige que a transformação aconteça dentro do ambiente analítico, via SQL.

Banco analítico: Banco otimizado para consultas que cruzam, [agregam](#) e filtram grandes volumes de dados (ver [OLAP](#)), diferente de um banco que registra operações do dia a dia. Bancos analíticos normalmente seguem [esquemas estrela](#) com alta [desnormalização](#).

Agregação: Operação que combina várias linhas em um único valor-resumo, aplicando uma função sobre uma coluna (ex.: SUM, AVG, COUNT, MIN, MAX). É a base de qualquer consulta analítica: em vez de listar cada linha, você resume um grupo delas em um número (ex.: total de vendas, média de duração, contagem de pedidos). Normalmente vem acompanhada de GROUP BY, que define por qual critério as linhas são agrupadas antes de agregar (contraste com [window functions](#), que calculam algo parecido mas sem colapsar as linhas em uma só).

Banco transacional: Banco otimizado para registrar operações individuais de forma correta e rápida (ver [OLTP](#)), não para analisar grandes volumes de uma vez, a prioridade será dados íntegros havendo grande [normalização](#).

OLTP (Online Transaction Processing): Arquitetura voltada a registrar operações corretamente: tabelas [normalizadas](#), muitas escritas e atualizações. Não é o objetivo do desafio; a fonte dele possivelmente vem de um OLTP de corridas de táxi.

OLAP (Online Analytical Processing): Arquitetura voltada a analisar muitas operações e encontrar padrões: [fatos](#) e [dimensões](#), leituras, filtros e [agregações](#). É o que o desafio pede.

Normalização: Técnica de organizar dados em várias tabelas relacionadas, eliminando redundância: cada informação é guardada uma única vez e referenciada por [chave](#) a partir de outras tabelas. Reduz duplicação e risco de inconsistência, mas exige mais junções (JOINS) para reconstruir a informação completa. Padrão típico de bancos OLTP. Um sistema de pedidos normalizado tem duas tabelas: clientes(cliente_id, nome, cidade) e pedidos(pedido_id, cliente_id, valor). Pra saber a cidade de quem fez um pedido, é preciso um JOIN entre as duas. Em compensação, se o cliente mudar de cidade, só uma linha precisa ser atualizada (na tabela clientes) o dado existe em um único lugar.

Desnormalização: Técnica oposta à [normalização](#): repetir ou agrupar dados propositalmente em menos tabelas, aceitando alguma redundância, para reduzir a quantidade de junções necessárias e acelerar leituras analíticas. Comum em bancos [OLAP](#). A mesma informação do exemplo anterior, desnormalizada, viraria uma tabela só: pedidos(pedido_id, cliente_id, nome_cliente, cidade_cliente, valor). Toda consulta que precisa da cidade do cliente já a encontra ali mesmo, sem JOIN. O custo: se esse cliente fez 500 pedidos e muda de cidade, são 500 linhas pra atualizar (ou o dado fica desatualizado/inconsistente em algumas delas), é o trade-off clássico de desnormalização: leitura mais rápida, risco de inconsistência maior.

Modelo dimensional: Forma de organizar dados analíticos em tabelas de fato (eventos/[medidas](#)) e tabelas de dimensão (contexto descritivo), otimizada para leitura e [agregação](#).

Tabela de fato: Tabela central de um modelo dimensional, onde cada linha representa um evento ou transação mensurável (o "[grão](#)"), e as colunas numéricas são as [medidas](#). A tabela de fato possuirá eventos e [medidas](#) numa única granularidade além de uma série de [chaves](#) que a ligará as tabelas de dimensão.

Tabela de dimensão: Tabela que descreve o contexto de um evento registrado na fato: quem, o quê, onde, quando. Contém [atributos descritivos](#), conecta-se à fato por uma [chave](#), e normalmente tem muito menos linhas que a fato.

Termos da seção 02 : Base de dados e ambiente

DuckDB: Banco de dados analítico que roda embutido (sem servidor separado), consulta arquivos Parquet diretamente via SQL.

Parquet: Formato de arquivo colunar, binário, feito para leitura analítica eficiente. Não deve ser editado manualmente.

Idempotência / reprodutibilidade: Propriedade de um pipeline onde executá-lo várias vezes, com a mesma fonte de dados, sempre produz o mesmo resultado. Se a fonte mudar (ex.: adicionar um novo mês), o resultado pode mudar: isso não quebra a idempotência.

Termos da seção 03 : Escopo técnico obrigatório

Camada raw (bruta): Cópia fiel dos dados como chegaram da fonte, sem nenhuma transformação, mantida para auditoria e rastreabilidade.

Perfilamento (data profiling): Investigação sistemática dos dados: tipos, volume, nulos, distribuições, domínios, [cardinalidade](#), duplicidade, valores improváveis antes de decidir qualquer regra de limpeza.

Cardinalidade: Quantidade de valores distintos que uma coluna assume (ex.: Uma coluna de gênero normalmente terá baixíssima cardinalidade (M/F ou masculino/feminino/outro) enquanto ID terá muitas vezes um valor diferente para cada linha)

Camada de tratamento (staging): Etapa onde tipos, nomes, categorias e valores são padronizados e problemas de qualidade são tratados, sempre com justificativa registrada.

Grão (granularidade) da fato: Descrição exata do que uma linha da tabela fato representa. Toda medida da fato só é somável se for consistente com esse grão (ex.: se "uma linha = um pedido", isso deve valer para todas as linhas, e cada coluna deve se referir só a esse pedido; não faria sentido ter o total de vendas do mês inteiro misturado numa tabela onde a linha é um pedido).

Camada de consumo analítico: Consultas finais que respondem perguntas de negócio, construídas sobre o modelo dimensional, nunca direto sobre o arquivo bruto.

Termos da seção 04 : Requisitos de limpeza e modelagem

Medida (measure): Atributo numérico da fato que pode ser somado, calculado ou comparado em consultas analíticas.

Chave (key): Coluna que identifica um registro (chave natural, já existente na fonte) ou que substitui esse identificador por um valor gerado internamente (chave substituta / surrogate key).

Atributo descritivo: Coluna de texto/categoria que qualifica um registro e deve viver na dimensão, não repetida em cada linha da fato.

Dimensão role-playing (reutilizável): A mesma tabela de dimensão usada mais de uma vez na fato com papéis diferentes, evitando duplicar a lógica de tradução de código em cada consulta.

Unknown member (membro desconhecido): Linha padrão de uma dimensão usada quando um código não tem correspondência conhecida, para não descartar o registro da fato.

Termos da seção 06 : Apoio para pesquisa

Lookup table: Tabela de referência, legenda usada para traduzir um código em um valor legível.

Domain values (valores de domínio): Conjunto de valores válidos esperados para uma coluna categórica, sua cardinalidade.

Star schema (esquema estrela): Desenho de modelo dimensional onde uma fato central se conecta diretamente a várias dimensões, sem camadas intermediárias de normalização.

CTE (Common Table Expression): Bloco WITH nome AS (...) que nomeia uma subconsulta, tornando SQL complexo mais legível e reutilizável.

TRY_CAST: Conversão de tipo que retorna NULL em vez de erro quando o valor não pode ser convertido.

Window functions (funções de janela): Funções que calculam um valor sobre um conjunto de linhas relacionado à linha atual, sem colapsar o resultado numa única linha, ao contrário de GROUP BY.

Idempotent SQL pipeline: Pipeline cujos scripts podem ser reexecutados sem gerar duplicidade ou alterar o resultado, tipicamente via CREATE OR REPLACE TABLE em vez de INSERT INTO puro.

Incremental load x full load: Carregar apenas os dados novos desde a última execução (incremental) versus reprocessar toda a fonte do zero a cada execução (full/completa).

Data lineage (linhagem de dados): Capacidade de rastrear de onde cada dado veio e por quais transformações passou até chegar ao resultado final.

Rejected records (registros rejeitados): Registros identificados como inválidos durante a limpeza, mantidos (não apagados) e documentados separadamente para auditoria.

Reconciliation (reconciliação): Verificação de que totais/contagens do modelo final batem com os totais/contagens da fonte bruta.

Rollup / drill down: Rollup é subir de nível de detalhe ([agregar](#), ex.: de dia para mês); drill down é descer (ver o detalhe por trás de um número [agregado](#), ex.: de mês para dia). Movimento típico de exploração num modelo [OLAP](#).

Dimensão degenerada: [Atributo](#) que fica registrado na tabela [fato](#) mesmo sem uma dimensão própria, por não ter contexto descritivo relevante além de si mesmo, geralmente um identificador vindo direto da fonte.

Sequence (geração de ID): Mecanismo do banco que gera automaticamente números sequenciais únicos, comumente usado para criar chaves substitutas em dimensões.